ΣΥΣΤΗΜΑΤΑ ΔΙΑΧΕΙΡΙΣΗΣ ΚΑΙ ΑΝΑΛΥΣΗΣ ΔΕΔΟΜΕΝΩΝ

Εαρινό εξάμηνο 2024-2025

Πρώτη Σειρά Ασκήσεων

Άσκηση 1

a. Ο συνολικός αριθμός των τομέων στον δίσκο είναι:
8 επιφάνειες × 4096 ίχνη/επιφάνεια × 1024 τομείς/ίχνος = 2^3 x 2^{12} x 2^{10} = **2^{25} τομείς**
και επειδή το μέγεθος κάθε τομέα είναι 512 bytes, η χωρητικότητα του δίσκου είναι:
2^{25} τομείς x 512 bytes/τομέα = 2^{25} x 2^9 bytes = **2^{34} bytes**
Άρα μέγιστος αριθμός εγγραφών (με μέγεθος 256 bytes η καθεμία):
2^{34} / 2^8 = **2^{26} = 67108864 εγγραφές.**

b. Για να διαβάσουμε **ένα block** από τον δίσκο απαιτείται χρόνος:

Time = Seek Time + Rotation Delay + Transfer Time, όπου:

Seek Time = 10 ms

Rotation Delay = ο χρόνος για **μισή περιστροφή**:

60 10^3 ms	7200 περιστροφές	
Rotation Delay	1/2 περιστροφή	**Rotation Delay = 4,17 ms**

Transfer Time: Ο χρόνος που απαιτείται για να «περάσουν» τα sectors του block κάτω από τις κεφαλές. Για να υπολογίσουμε αυτόν τον χρόνο έχουμε:

1 block = 4096 bytes = 8 x 512 bytes => **1 block = 8 sectors**

Σε 1 λεπτό (60 x 10^3 ms) γίνονται 7200 περιστροφές, άρα η κάθε περιστροφή διαρκεί 60 x 10^3 /7200 ms = 8,33 ms. Στον χρόνο αυτό της μίας περιστροφής, περνάνε κάτω από την κεφαλή ανάγνωσης όλοι οι sectors του track οι οποίοι είναι 1024 και άρα (επειδή 1 sector = 1/8 block) περνάνε 1024/8= $2^{10}/2^3$ = 2^7 blocks. Έτσι για να περάσει ένα block κάτω από την κεφαλή ανάγνωσης χρειάζεται χρόνο 8,33/128 ms.

Έτσι **Transfer Time = 0,07 ms**.

Οπότε ο συνολικός χρόνος για να διαβάσουμε ένα τυχαίο block είναι Time = Seek Time + Rotation Delay + Transfer Time = 10 + 4,17 + 0,07 ≈ 14,24 ms και έτσι **για να διαβάσουμε Χ τυχαία block θα θέλουμε χρόνο 14,24Χ ms**

Αν διαβάζαμε όλο το αρχείο σειριακά δε έχουμε Rotation Delay (=0) και έχουμε μόνο ένα Seek Time (10 ms).

Το αρχείο έχει 1000000 εγγραφές μεγέθους 256 bytes η καθεμία, άρα έχει 1000000 x 256 bytes. Το κάθε block έχει 4096 bytes, άρα όλο το αρχείο αποτελείται από:

1000000 x 256 bytes / 4096 bytes/block = 1000000/2^4 blocks = **62500 blocks** και επειδή το κάθε block χρειάζεται 0,07 ms για να διαβαστεί, όλο το αρχείο έχει Transfer Time = 62500 x 0,07 => Transfer Time = 4375 ms.

Έτσι ο συνολικός χρόνος για να διαβαστεί όλο το αρχείο είναι Time = Seek Time + Rotation Delay + Transfer Time = 10 + 0 + 4375 ms => Time = **4385 ms**.

Για να βρούμε το κρίσιμο πλήθος εγγραφών όπου οι δύο χρόνοι είναι ίσοι, έχουμε:

14,24X = 4385 => X = 4385/14,24 => **X ≈ 308**

Δηλαδή, όταν πρέπει να αναζητήσουμε περισσότερες από 308 τυχαίες εγγραφές, είναι πιο αποδοτικό να διαβάσουμε ολόκληρο το αρχείο.

Άσκηση 2

a. Το μέγεθος της κάθε εγγραφής είναι: 6+25+25+8=64 bytes . Το μέγεθος του μπλοκ (σελίδας) είναι 512 bytes και άρα κάθε μπλοκ χωράει 512/64 = 8 εγγραφές. Επειδή έχουμε 1000000 εγγραφές θα χρειαστούμε για την αποθήκευση του αρχείου: 100000 / 8 = **12500 μπλοκ**.

b. Για να υπολογίσουμε τον μικρότερο και τον μεγαλύτερο αριθμό των μπλοκ που θα καταλαμβάνει το δευτερεύον B+ ευρετήριο, υπολογίζουμε τα εξής:

Το μέγεθος της εγγραφής κάθε φύλλου του ευρετηρίου = Μέγεθος κλειδιού ευρετηρίου (γνώρισμα D) + Μέγεθος δείκτη δεδομένων = 8 + 5 = **13 bytes**.

Ο αριθμός εγγραφών ευρετηρίου ανά μπλοκ = $\lfloor$Μέγεθος μπλοκ / Μέγεθος εγγραφής φύλλου$\rfloor$ = $\lfloor 512/13 \rfloor$= 39, δηλαδή **χωρούν 39 εγγραφές ανά μπλοκ**.

Ο αριθμός των φύλλων του ευρετηρίου είναι ίσος με τον αριθμό των εγγραφών ευρετηρίου, δηλαδή 100000. Έτσι ο αριθμός μπλοκ των φύλλων = $\lceil 100000 / 39 \rceil$ = 2565 . Άρα, χρειάζονται **2565 μπλοκ για τα φύλλα**.

Στη συνέχεια υπολογίζουμε τον αριθμό των μπλοκ για τους εσωτερικούς κόμβους (αυτοί που δεν είναι φύλλα) ως εξής:

Κάθε εγγραφή αυτού του κόμβου έχει μέγεθος = Μέγεθος κλειδιού ευρετηρίου (γνώρισμα D) + Μέγεθος δείκτη προς κόμβο = 8+4=**12 bytes**. Οπότε ο αριθμός εγγραφών ανά μπλοκ = $\lfloor 512/12 \rfloor$ = 42, δηλαδή **χωρούν 42 εγγραφές ανά μπλοκ**.

Όπως είδαμε πριν, στο B+ δέντρο υπάρχουν 2565 φύλλα, άρα στο προηγούμενο (πιο πάνω) επίπεδο χρειάζονται $\lceil 2565/42 \rceil$ = **62 μπλοκ**. Ο αριθμός των μπλοκ στο προηγούμενο (πιο πάνω) επίπεδο είναι $\lceil 62/42 \rceil$ = **2 μπλοκ** και η ρίζα του δένδρου χρειάζεται **1 επιπλέον μπλοκ.**

Άρα στην καλύτερη περίπτωση (μπλοκ πλήρως γεμάτα) χρειαζόμαστε 2565 + 62 + 2 + 1 = 2630 μπλοκ.

Στη χειρότερη περίπτωση, τα φύλλα του B+ δέντρου είναι μισογεμάτα (λόγω εισαγωγών/διαγραφών) άρα, κάθε μπλοκ περιέχει περίπου τη μισή χωρητικότητα δηλαδή $\lceil 39/2 \rceil =$ **20 εγγραφές ανά μπλοκ**. Άρα χρειάζονται $\lceil 10000/20 \rceil$ = **5000 μπλοκ για τα φύλλα**.

Με το ίδιο σκεπτικό όπως προηγούμενα, κάθε εσωτερικός κόμβος θα χωράει $\lceil 42/2 \rceil$ = 21 εγγραφές ανά μπλοκ. Οπότε στο πιο πάνω από τα φύλλα επίπεδο χα έχουμε $\lceil 5000/21 \rceil$ = **239 μπλοκ**. Ο αριθμός των μπλοκ στο προηγούμενο επίπεδο θα είναι $\lceil 239/21 \rceil$ = **12 μπλοκ** και η ρίζα του δέντρου χρειάζεται ακόμη **1 μπλοκ**.

Άρα στην χειρώτερη περίπτωση (μπλοκ μισογεμάτα) χρειαζόμαστε 5000 + 239 + 12 + 1 = 5252 μπλοκ.

c. Όπως προέκυψε από τους υπολογισμούς στο ερώτημα b, το μικρότερο B+ δέντρο έχει 3 επίπεδα εκτός των φύλλων (άρα ύψος 3 από τη ρίζα μέχρι το επίπεδο αμέσως πάνω από τα φύλλα). Οπότε θα χρειαστεί να διαβάσουμε 3 μπλοκ για να φτάσουμε στα φύλλα. Επειδή θέλουμε να ανακτήσουμε 1000 εγγραφές, και κάθε φύλλο περιέχει 39 εγγραφές (όπως είδαμε στο ερώτημα b), θα πρέπει να διαβάσουμε $\lceil 1000/39 \rceil$ = 26 μπλοκ φύλλων. Άρα για την διάσχιση του ευρετηρίου θα χρειαστούμε 3+26=29 μπλοκ.

Για κάθε μία από τις 1000 εγγραφές που βρέθηκαν στο ευρετήριο, πρέπει να διαβάσουμε το αντίστοιχο μπλοκ δεδομένων από τη σχέση R. Δεδομένου ότι το ευρετήριο είναι δευτερεύον (non-clustered), οι εγγραφές αυτές μπορεί να βρίσκονται σε διαφορετικά μπλοκ. Στη χειρότερη περίπτωση, κάθε εγγραφή βρίσκεται σε διαφορετικό μπλοκ. Έτσι, πρέπει να διαβαστούν 1000 μπλοκ δεδομένων, έτσι:

Συνολικός αριθμός μπλοκ που πρέπει να διαβαστούν = Μπλοκ ευρετηρίου + Μπλοκ δεδομένων = 29 + 1000 = 1029 μπλοκ.

Άσκηση 3

a. Από το σχήμα παρατηρούμε ότι το ολικό βάθος είναι 2 (i = 2).

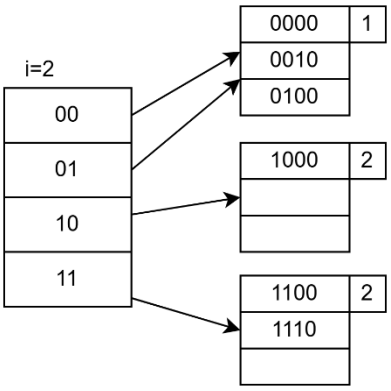
Ο κάδος για κλειδιά που αρχίζουν με 0 έχει τοπικό βάθος 1 και είναι και ο μόνος που μπορεί να σε "σπάσει" σε δύο κάδους τοπικού βάθους 2 (για 00 και 11), χωρίς να αυξηθεί το ολικό βάθος, μιας και το τοπικό βάθος ήταν μικρότερο από το ολικό. Έτσι λοιπόν θα έχουμε 4 κάδους τοπικού βάθους 2 (μετά το σπάσιμο του κάδου 0), καθένας με χωρητικότητα 3 τιμών. Άρα ο μέγιστος αριθμός τιμών που μπορεί να χωρέσει το ευρετήριο δίχως να αυξηθεί το ολικό του βάθος είναι 4×3 = **12**. Αν θεωρήσουμε ότι όπως φαίνεται στο σχήμα υπάρχουν ήδη 6 τιμές, θα μπορούσαμε να προσθέσουμε το πολύ άλλες **6** δίχως να αυξηθεί το ολικό του βάθος, αρκεί αυτές οι τιμές να ήταν κατάλληλες (οι 2 από αυτές να άρχιζαν από 10, μία να άρχιζε με 11, μία να άρχιζε με 00 και δύο να άρχιζαν με 01).

b. Για να αυξηθεί το ολικό βάθος (δηλαδή να διπλασιαστεί το ευρετήριο) απαιτείται να "σπάσει" ένας κάδος του οποίου το τοπικό βάθος είναι ήδη ίσο με το ολικό βάθος. Στο παράδειγμά μας το ολικό βάθος είναι i=2 και υπάρχουν δύο κάδοι με τοπικό βάθος 2 (αυτοί που περιέχουν τιμές που αρχίζουν με 10 και 11). Κάθε κάδος χωράει 3 τιμές, επίσης όπως είδαμε και προηγουμένως, κάδος που περιέχει τιμές που αρχίζουν με 0, έχει τοπικό βάθος 1 (και άρα, αν γεμίσει, θα σπάσει σε τοπικό βάθος 2 χωρίς να αυξηθεί το ολικό βάθος). Θα χρειαστεί διπλασιασμός του ευρετηρίου, μόνο αν προσπαθήσουμε να εισάγουμε 4η τιμή σε κάποιον κάδο τοπικού βάθους 2.

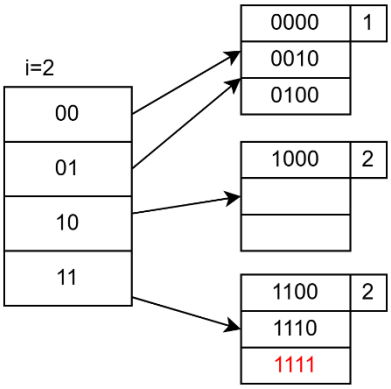
Στο συγκεκριμένο σχήμα φαίνεται ότι ο κάδος που δέχεται τιμές που αρχίζουν από 11, έχει μία ελεύθερη θέση οπότε θα μπορούσε να δεχθεί μία ακόμη τιμής που αρχίζει με 11, χωρίς να υποστεί υπερχείλιση, όμως μετά αν προσπαθούσαμε να βάλουμε στο ευρετήριο άλλη μία τιμή που αρχίζει με 11, αυτή θα πρέπει να πάει στον ίδιο κάδο προκαλώντας διπλασιασμό του ευρετηρίου. Άρα στο συγκεκριμένο παράδειγμα θα αρκούσαν **2** τιμές για να προκαλέσουν διπλασιασμό του ευρετηρίου (π.χ. 1111 και 1101).

c. Εισαγωγή τιμών:

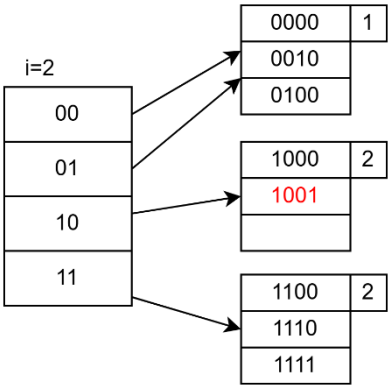
Αρχικά:



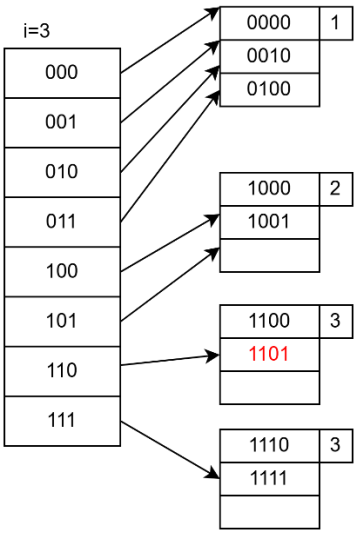
Εισαγωγή του 1111:



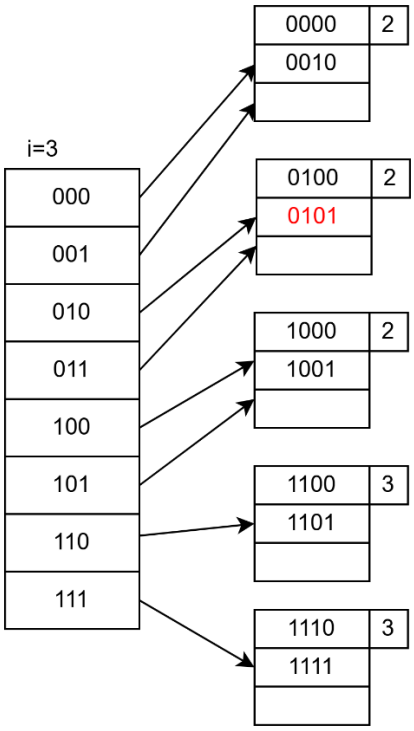
Εισαγωγή του 1001:



Εισαγωγή του 1101: θα έπρεπε να μπει στον κάδο που δέχεται τιμές που ξεκινούν με 11, όμως επειδή αυτός είναι ήδη γεμάτος, έχουμε διάσπαση αυξάνοντας το τοπικό βάθος κατά 1. Παρατηρούμε ότι το νέο τοπικό βάθος είναι 3 και είναι μεγαλύτερο από το ολικό, οπότε θα αυξηθεί και το ολικό βάθος και το ευρετήριο διπλασιάζεται:



Εισαγωγή του 0101: θα έπρεπε να μπει στο κάδο που δέχεται τιμές που ξεκινούν με 0, όμως επειδή αυτός είναι γεμάτος, έχουμε διάσπαση αυξάνοντας το τοπικό βάθος κατά 1 και από 1 γίνεται 2:

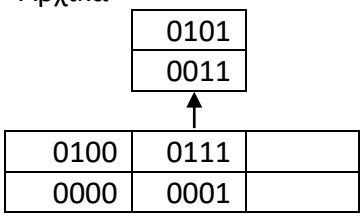


Άσκηση 4

a. Το **i = 2** (για την κατανομή των τιμών χρησιμοποιούνται τα 2 λιγότερο σημαντικά bits) και **το m = $(10)_2$ = 2** (max used block).

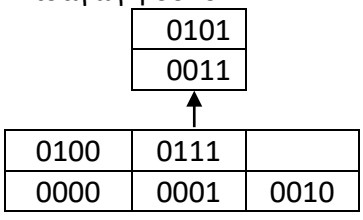
b. Εισαγωγή τιμών:

Αρχικά



Utilization 6/8 = 0,75 i=2 m=2(10)

Εισαγωγή 0010



Utilization 7/8 = 0,875 οπότε το m αυξάνεται σε 3(11)

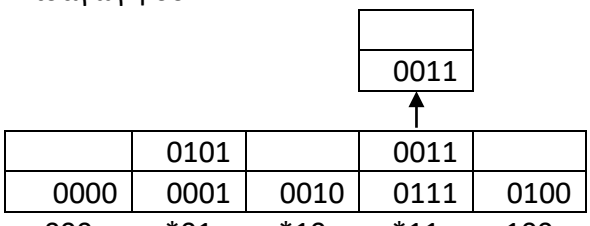
0100	0101		0011
0000	0001	0010	0111
00	01	10	11

Utilization 7/8 = 0,875 οπότε το i αυξάνεται σε 3 και το m αυξάνεται σε 4(100)

	0101		0011	
0000	0001	0010	0111	0100
000	*01	*10	*11	100

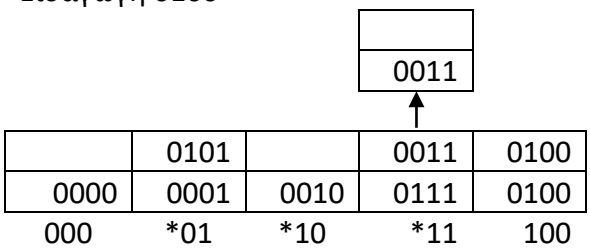
Utilization 7/10 = 0,7 i = 3 m = 4(100)

Εισαγωγή 0011



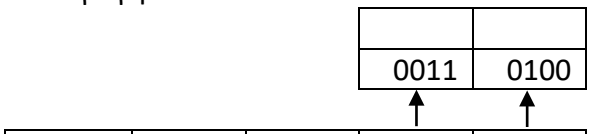
Utilization 8/12 = 0,67 i = 3 m = 4(100)

Εισαγωγή 0100



Utilization 9/12 = 0,75 i = 3 m = 4(100)

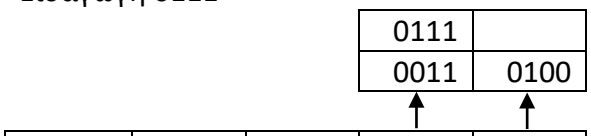
Εισαγωγή 0100



	0101		0011	0100
0000	0001	0010	0111	0100
000	*01	*10	*11	100

Utilization 10/14 = 0,71 i = 3 m = 4(100)

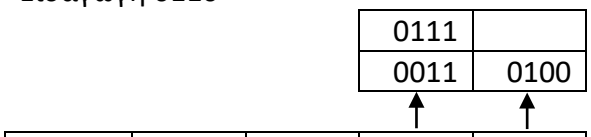
Εισαγωγή 0111



	0101		0011	0100
0000	0001	0010	0111	0100
000	*01	*10	*11	100

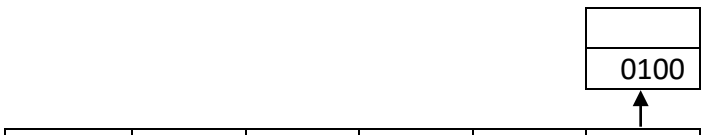
Utilization 11/14 = 0,79 i = 3 m = 4(100)

Εισαγωγή 0110



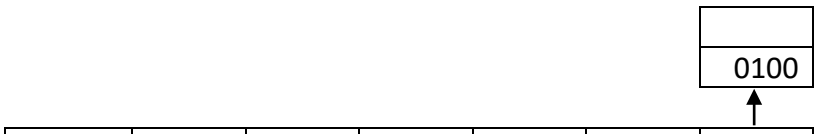
	0101	0110	0011	0100
0000	0001	0010	0111	0100
000	*01	*10	*11	100

Utilization 12/14 = 0,86 οπότε το m αυξάνεται σε 7(111)



	0101	0110	0111	0011	0100
0000	0001	0010	0111	0011	0100
000	*01	*10	111	011	100

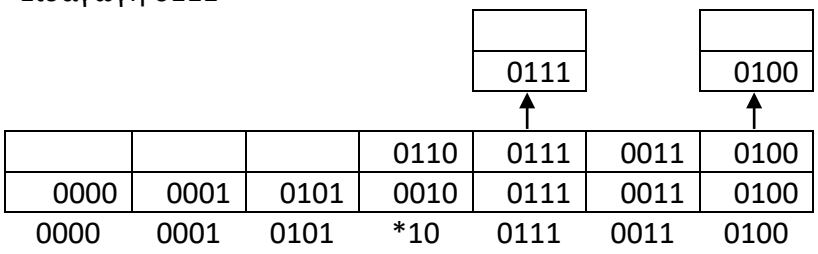
Utilization 12/14 = 0,86 οπότε το i αυξάνεται σε 4



			0110	0111	0011	0100
0000	0001	0101	0010	0111	0011	0100
0000	0001	0101	*10	0111	0011	0100

Utilization 12/16 = 0,75 i = 4 m = 7(111)

Εισαγωγή 0111

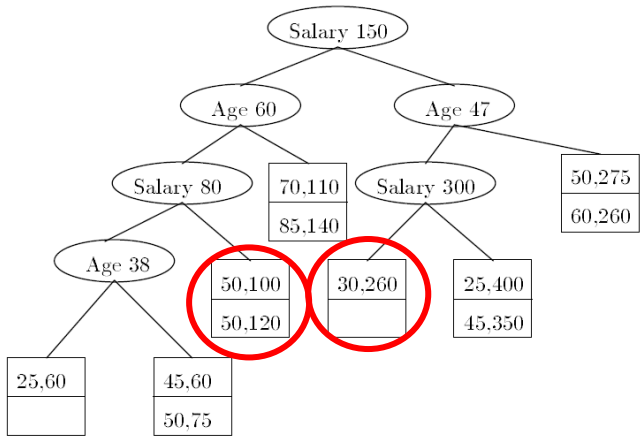


Utilization 13/18 = 0,72 i = 4 m = 7(111)

c. Στο ευρετήριο αυτή τη στιγμή υπάρχουν 13 κλειδιά αποθηκευμένα σε 7 κάδους και 2 overflow pages. Έτσι όπως είναι αποθηκευμένα τα κλειδιά και λαμβάνοντας υπόψη ότι αν μια εγγραφή βρίσκεται σε κάδο χρειάζεται μία προσπέλαση για την ανάκτησή της, ενώ αν βρίσκεται σε overflow page χρειάζεται δύο προσπελάσεις, έχουμε ότι 11 από τις 13 εγγραφές χρειάζονται 1 προσπέλαση ενώ 2 από τις 13 χρειάζονται 2 προσπελάσεις, άρα ο μέσος αριθμός προσπελάσεων είναι

(1+1+1+1+1+1+1+1+1+1+1+2+2)/13 = 15/13 = **1,15**

Άσκηση 5



α) Φύλλο με τιμές (30,260)

Από το διάγραμμα βλέπουμε ότι για να φτάσουμε στο φύλλο με (Age=30, Salary=260) ακολουθούμε την διαδρομή:

Ρίζα (Salary 150), πάμε δεξιά, άρα Salary>=150. Επόμενος κόμβος (Age 47) πάμε αριστερά, άρα Age<47. Επόμενος κόμβος: (Salary 300), πάμε αριστερά, άρα Salary<300. Δεν υπάρχει άλλη διάσπαση, άρα το συγκεκριμένο φύλλο (που έχει το (30,260)) δέχεται όλα τα σημεία (Age, Salary) που ικανοποιούν τις εκφράσεις:

150 <= Salary <300 AND Age < 47

β) Φύλλο με τιμές (50,100) και (50, 120)

Ακολουθώντας την ίδια λογική όπως και προηγουμένως, καταλήγουμε ότι το συγκεκριμένο φύλλο δέχεται όλα τα σημεία (Age, Salary) που ικανοποιούν τις εκφράσεις:

80 <= Salary < 150 AND Age < 60